

ECE-510: Hardware for AI and ML, Spring-2025

High Level Design Specification (HLDS) for Q-Learning Hardware-Software Integrated Agent

By Jaswanth Pallappa

1. Introduction

1.1 Purpose

This document outlines the high-level design specification for a hybrid Q-learning agent. The project implements a software-hardware co-design where the computationally intensive Q-value update logic is offloaded to hardware using custom SystemVerilog RTL. The learning logic, state transitions, and environment simulation are managed in Python.

1.2 Document Conventions

All hardware modules are implemented in SystemVerilog. Python (NumPy, Matplotlib) is used for environment modeling, agent control, and visualization. Profiling is conducted using cProfile and PyVerilator is used for integration testing.

1.3 Intended Audience

- Hardware Design Engineers
- Digital Logic Designers
- Reinforcement Learning Researchers
- Students learning AI hardware acceleration

1.4 Product Scope

The project builds a deterministic Frozen Lake Q-learning agent and offloads the Q-value update rule to an RTL hardware block using Wallace Tree Multipliers, Brent-Kung Adders, and Barrel Shifters for performance. Python handles learning flow, and Verilog accelerates arithmetic computation.

1.5 References

- Sutton & Barto, Reinforcement Learning: An Introduction
- IEEE 754 Standard
- PyVerilator Docs
- Brent-Kung Adder Architecture
- Wallace Tree Multiplication Method

2. Overall Description

2.1 Project Perspective

This hybrid project blends Python-based control logic with RTL modules responsible for updating Q-values using the Bellman equation. Integration is done using PyVerilator.

2.2 Product Functions

- Software Grid Environment and Agent (Python)
- Hardware Q-Value Updater (Verilog)
- Reward and Transition Logic (Python)
- Learning Visualization (Matplotlib)
- Time Profiling (cProfile)

2.3 User Classes

- AI Hardware Engineers
- Embedded Systems Developers
- Research Students

2.4 Tools and Software

- Python 3.11, NumPy, Matplotlib
- PyVerilator
- QuestaSim 2023.4 (for simulation)
- OpenLane (for FPGA synthesis)

2.5 Design Constraints

- Fixed-point arithmetic (Q4.12)
- RTL computation latency must be < 2 clock cycles for critical path
- Real-time Q-update throughput target: $> 1\text{M}$ updates/sec

2.6 Assumptions

- All inputs are 32-bit fixed point.
- Python testbench can pause and feed inputs to RTL through DPI or file I/O

3. External Interface Requirements

3.1 Hardware Interfaces

- Q-Update Module: Inputs: `q_sa`, `reward`, `max_q_spap`, `alpha`, `gamma`; Output: `q_new`
- Synchronized on `clk`, `rst`

4. Product Features

4.1 Q-Value Update Accelerator (RTL)

- Wallace Tree Multiplier
- Brent-Kung Adder/Subtractor
- Barrel Shifter (Right)
- Fixed-point arithmetic (Q4.12)

4.2 Software Agent & Environment (Python)

- 10x10 Grid environment
- Start, Win, and Hole states
- Epsilon-greedy action policy
- Reward tracking and plotting

5. Logic Design

5.1 Directory Structure

```
qlearning_hw/  
├── python/                # Agent & Training loop  
│   └── q_agent.py  
├── rtl/                   # RTL modules  
│   ├── brent_kung_adder.v  
│   ├── brent_kung_subtractor.v  
│   ├── wallace_multiplier.v  
│   ├── barrel_shifter.v  
│   └── q_update_core.v  
├── verification/          # Testbenches  
│   └── q_update_tb.sv  
└── docs/                  # HLDS and profiling results
```

5.2 Design Modules

- `q_update_core.v`: Integrates submodules to compute Bellman Q-update
- `brent_kung_adder.v`, `subtractor.v`: Low-latency prefix trees
- `wallace_multiplier.v`: Parallel partial product tree
- `barrel_shifter.v`: Single-cycle right shift

5.3 SV Abstractions

- Parameters used for WIDTH
- Blocking/non-blocking assignments
- RTL registers for output

5.4 Tools

- Simulation: QuestaSim, PyVerilator
- Synthesis: Vivado or Quartus
- Profiling: time, cProfile

6. Verification

6.1 Testbench Style

- SystemVerilog testbench for `q_update_core.v`
- Python testbench integration with PyVerilator

6.2 Testing Strategy

- Golden model in Python
- Fixed-point match within 0.5% tolerance

6.3 Test Cases

- `q_sa=0.25`, `reward=-1`, `max_q=0.5`, `alpha=0.5`, `gamma=0.9`
- Edge cases: overflow, underflow, zero input

6.4 Corner Cases

- Multiple Q-values at same max
- Zero or negative rewards
- Q-value saturation

Summary

This document defines a hardware-software Q-learning integration project using a modular hardware accelerator for Q-value updates. Python orchestrates learning while Verilog handles computation, accelerating reinforcement learning training loops for real-time or embedded deployment.